

Task 10.3HD: Complete Awesome DevOps Feature(s)

2026/T2 - Part B: Implementation Walkthrough and Evidence

Source code repository	https://github.com/JasSin155/koalatech-hd
GitOps repository (Argo CD)	https://github.com/JasSin155/koalatech-gitops
Narrated video (Deakin Panopto)	https://deakin.au.panopto.com/Panopto/Pages/Viewer.aspx?id=9472cf80-b10e-4df7-8fc8-b4d70009261f

This document explains how I built the two features, in the same eight steps as the video, so that another developer could repeat the work from my repositories. For each step I describe the problem it solves, the configuration I wrote, why I made the choices I did, and what the evidence shows. Every screenshot was captured from the live environment during this task, on the CloudLabs subscription, on 1 October 2026.

1) Design: What I Changed and Why

The pipeline I finished in Task 10.2D was a push pipeline. GitHub Actions held an Azure service principal secret, logged in to the cluster, and ran `kubectl apply`. That works, but it means the CI system is the most privileged thing in the whole design, and once a deployment finishes, nothing notices if the cluster drifts away from what was deployed. Table 1 summarises how each pattern from the unit was replaced.

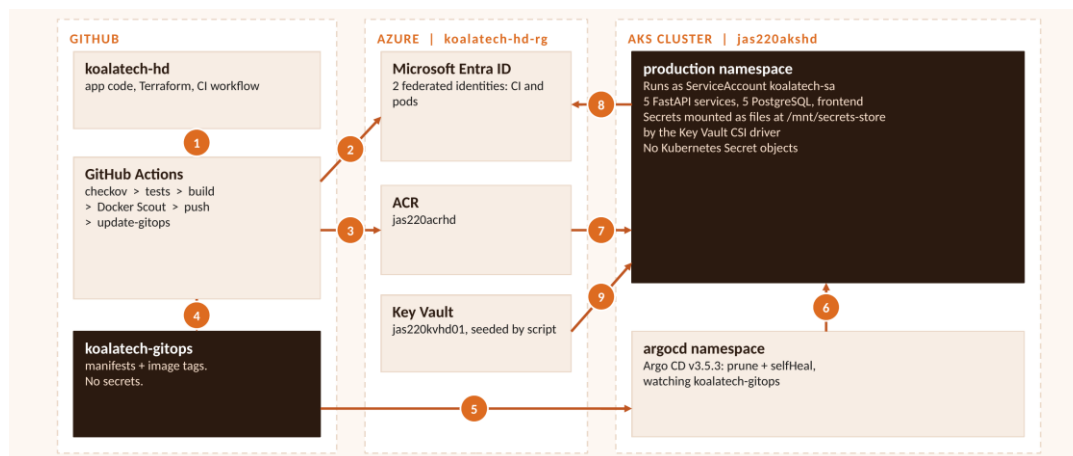


Figure 1: The implemented architecture. The numbers trace one change from a git push to a running pod, and the only link between CI and the cluster is a Git commit.

Concern	What the unit taught (Weeks 6 to 10)	What I built for 10.3HD
Deployment	CI pushes with <code>kubectl apply</code> , once per run	Argo CD pulls from a GitOps repository and reconciles continuously
Secrets	GitHub secret, then a base64 Kubernetes Secret	Azure Key Vault, mounted as files by the CSI driver, no Kubernetes Secret
Pod identity	None, credentials passed in as environment variables	Workload Identity, a federated managed identity bound to one ServiceAccount
CI identity	AZURE_CREDENTIALS client secret with Contributor	GitHub OIDC to a managed identity with <code>AcrPush</code> on one registry only
Security scanning	Run by hand or reported, never blocking	checkov and Docker Scout as required checks enforced by a branch ruleset

Table 1. Each weakness from the Week 10 pipeline and the pattern that replaced it.

Figure 1 shows the result. It has three zones, GitHub on the left, Azure platform services in the middle, and the AKS cluster on the right, and the numbered arrows follow one code change from my laptop to a running pod:

1. I push application code to the koalatech-hd repository.
2. GitHub Actions requests an OIDC token and exchanges it with Entra ID for an Azure token. No password is stored anywhere.
3. The pipeline runs checkov on the Terraform and Docker Scout on each image. If either fails, the run stops here.
4. The scanned image is pushed to Azure Container Registry using the AcrPush role only.
5. The last CI job commits the new image tag to the koalatech-gitops repository. This is the only thing CI changes, and it never contacts the cluster.
6. Argo CD, running inside the cluster, detects the new commit and syncs the production namespace to it.
7. The AKS nodes pull the new image from ACR through the kubelet identity's AcrPull role.
8. Each pod presents its signed ServiceAccount token to Entra ID and receives an Azure token for the workload identity.
9. The CSI driver uses that token to read Key Vault and mounts each secret as a file inside the pod.

The key design property is that the arrows never run from GitHub into the cluster. The cluster only ever reaches out, to Git and to Azure, so a compromised pipeline can at worst commit a bad image tag, which is visible in Git history and can be reverted, and it can never read a secret or run a command in production.

2) Step 1: Preparing Azure and Provisioning the Platform with Terraform

Before Terraform runs, `scripts/prep-azure.sh` creates the resource group and grants my own account User Access Administrator on that resource group only. I learnt in Weeks 6 to 10 that the CloudLabs Contributor role cannot create role assignments, and this design needs two of them, AcrPull for the cluster and AcrPush for the CI identity. Pre-creating the resource group and scoping the extra role to it means `terraform apply` completes in one pass, rather than failing half way through as it did in earlier tasks. Terraform then reads the resource group as a data source rather than owning it.

The most important part of the module is the AKS cluster. Three settings make the rest of the design possible, and I placed all three in Terraform so that nobody has to remember a manual step:

```
resource "azurerm_kubernetes_cluster" "aks" {
  ...
  # The cluster becomes an OIDC token issuer, and the mutating webhook
  # projects a signed ServiceAccount token into labelled pods.
  oidc_issuer_enabled      = true
  workload_identity_enabled = true

  # Secrets Store CSI driver + Azure Key Vault provider, managed by Terraform
  key_vault_secrets_provider {
    secret_rotation_enabled = true
    secret_rotation_interval = "2m"
  }
}
```

Excerpt from `terraform/kubernetes_service.tf`.

The OIDC issuer lets the cluster sign a short lived token for each pod, Workload Identity injects that token into any pod labelled for it, and the `key_vault_secrets_provider` block installs the Secrets Store CSI driver as a managed AKS add-on. My original design for this task enabled the add-on afterwards with `az aks enable-addons`, which was exactly the kind of manual step that is forgotten when the environment is rebuilt, so I moved it into Terraform. Secret

rotation polls Key Vault every two minutes, so a rotated value reaches the mounted file without redeploying the pod.

```
(and one more similar warning elsewhere)

Apply complete! Resources: 12 added, 0 changed, 0 destroyed.

Outputs:
acr_login_server = "jas220acrhd.azurecr.io"
acr_name = "jas220acrhd"
aks_cluster_name = "jas220akshd"
aks_oidc_issuer_url = "https://australiaeast.oic.prod-aks.azure.com/2625129d-99a2-4df5-988e-5c5d07e7d0fb/035959c1-af4d-4475-b171-51e0b28"
github_actions_client_id = "24f23869-86e2-4c9f-9b9f-c9a576f1971c"
key_vault_name = "jas220kvhd01"
resource_group_name = "koalatech-hd-rg"
storage_account_name = "jas220sthd"
subscription_id = "e91bb219-e696-48b8-af87-76efdba28491"
tenant_id = "2625129d-99a2-4df5-988e-5c5d07e7d0fb"
workload_identity_client_id = "2acb228b-67dd-4f8a-9021-01ef1ab89a3a"

jassi@jas_PC MINGW64 ~/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech-hd/terraform
```

Figure 2: terraform apply creating 12 resources, followed by the outputs that the seed script, the GitOps manifests and the GitHub variables are built from.

Figure 2 shows the apply completing with 12 resources added and nothing changed or destroyed. The outputs matter because they are the only values that later steps need, and none of them is a secret. The OIDC issuer URL is public by design, and the client and tenant IDs are identifiers rather than credentials, which is why I could later store them as GitHub variables rather than secrets.

☐ Name ↑	Type	Location
☐  jas220acrhd	... Container registry	Australia East
☐  jas220akshd	... Kubernetes service	Australia East
☐  jas220kvhd01	... Key vault	Australia East
☐  jas220sthd	... Storage account	Australia East
☐  koalatech-hd-github-id	... Managed Identity	Australia East
☐  koalatech-hd-workload-id	... Managed Identity	Australia East

☐ Name ↑	Type	Location
☐  7af51243-5280-4f7e-b737-b3e7f503317f	... Public IP address	Australia East
☐  aks-agentpool-17537846-nsg	... Network security group	Australia East
☐  aks-default-20668956-vmss	... Virtual machine scale set	Australia East
☐  aks-vnet-17537846	... Virtual network	Australia East
☐  azurekeyvaultsecretsprovider-jas220akshd	... Managed Identity	Australia East
☐  jas220akshd-agentpool	... Managed Identity	Australia East
☐  kubernetes	... Load balancer	Australia East

Figure 3: Top: the resource group Terraform built, with the registry, cluster, vault, storage account and two managed identities. Bottom: the AKS node resource group, containing azurekeyvaultsecretsprovider-jas220akshd.

The top half of Figure 3 shows the six resources in koalatech-hd-rg, including two separate managed identities, one for the pods and one for GitHub Actions, so that each holds only the permissions it needs. The bottom half is the evidence that the CSI add-on was enabled by Terraform. AKS only creates the azurekeyvaultsecretsprovider identity in the node resource group when the add-on is turned on, and I never ran an add-on command by hand.

3) Step 2: Federated Identity for the Pods and for GitHub Actions

A token signed by the cluster is only useful if Entra ID agrees to trust it. That trust is a federated identity credential, which says, in effect, accept tokens from this issuer, for this exact subject, in exchange for an access token for this managed identity. I created two of them.

```
resource "azurerm_federated_identity_credential" "workload" {
  name      = "koalatech-k8s-sa"
  parent_id = azurerm_user_assigned_identity.workload.id
  audience  = ["api://AzureADTokenExchange"]
  issuer     = azurerm_kubernetes_cluster.aks.oidc_issuer_url
  subject    = "system:serviceaccount:production:koalatech-sa" # via variables
}

resource "azurerm_federated_identity_credential" "github_main" {
  name      = "github-koalatech-hd-main"
  parent_id = azurerm_user_assigned_identity.github_actions.id
  audience  = ["api://AzureADTokenExchange"]
  issuer     = "https://token.actions.githubusercontent.com"
  subject    = "repo:${owner}@${owner_id}/${repo}@${repo_id}:ref:refs/heads/main"
}
```

Excerpt from `terraform/workload_identity.tf`, with the variable interpolation shortened for readability.

The first credential trusts only the koalatech-sa ServiceAccount in the production namespace. A pod in any other namespace, or running as any other ServiceAccount, receives nothing from Azure, even if it sits on the same cluster. The second replaces the client secret that every earlier task stored in GitHub. Only a workflow running on the main branch of this exact repository can exchange its GitHub token for the CI identity, and that identity holds a single role, AcrPush on one registry. Pull requests never receive Azure access at all, they only build and scan.

Repository variables			New repository variable
Name	Value	Last updated	
ACR_LOGIN_SERVER	jas220acrhd.azurecr.io	now	
AZURE_CLIENT_ID	24f23869-86e2-4c9f-9b9f-c9a576f1971c	now	
AZURE_SUBSCRIPTION_ID	e91bb219-e696-48b8-af87-76efdba28491	now	
AZURE_TENANT_ID	2625129d-99a2-4df5-988e-5c5d07e7d0fb	now	

Repository secrets			New repository secret
Name	Last updated		
DOCKERHUB_TOKEN	now		
DOCKERHUB_USERNAME	now		
GITOPS_TOKEN	now		

Figure 4: Top: GitHub repository variables, which hold only identifiers. Bottom: repository secrets, which contain no Azure credential, only the Docker Hub token for Docker Scout and the GITOPS_TOKEN used to commit to the GitOps repository.

Figure 4 shows the result on the GitHub side. The four Azure values are variables, not secrets, because none of them grants access by itself. Compared with Task 10.2D, the AZURE_CREDENTIALS secret is gone. The two secrets that remain are deliberately narrow, a Docker Hub token used only to authenticate Docker Scout, and a fine grained token that can write to the one GitOps repository.

What went wrong first, and why the fix made the design stricter

```

1 ▶ Run azure/login@v2
14 Running Azure CLI Login.
15 /usr/bin/az cloud set -n azurecloud
16 Done setting cloud: "azurecloud"
17 Federated token details:
18 issuer - https://token.actions.githubusercontent.com
19 subject claim - repo:JasSin155@280837182/koalatech-hd@1398931302:ref:refs/heads/main
20 audience - api://AzureADTokenExchange
21 job_workflow_ref - JasSin155/koalatech-hd/.github/workflows/ci.yml@refs/heads/main
22 Attempting Azure CLI login by using OIDC...
23 Error: AADSTS700213: No matching federated identity record found for presented assertion subject 'repo:JasSin155@280837182/koalatech-hd@1398931302:ref:refs/heads/main'. Check your federated identity credential Subject, Audience and Issuer against the presented assertion.
    https://learn.microsoft.com/entra/workload-id/workload-identity-federation Trace ID: 3241017d-2a98-4e45-a4d6-6c7977313400 Correlation ID: 047c5147-93fb-481a-9846-02578f2c03f9 Timestamp: 2026-10-01 02:59:04Z
24
25 Error: Run the command below to authenticate interactively; additional arguments may be added as needed:
26 az logout
27 az login
28
29 Error: Login failed with Error: The process '/usr/bin/az' failed with exit code 1. Double check if the 'auth-type' is correct. Refer to
    https://github.com/Azure/login#readme for more information.

```

Figure 5: The first pipeline run failing with AADSTS700213. The subject claim GitHub presented includes the numeric owner ID 280837182 and repository ID 1398931302.

My first version used the documented subject format, `repo:JasSin155/koalatech-hd:ref:refs/heads/main`. Figure 5 shows the first run rejecting it with AADSTS700213, no matching federated identity record. Reading line 19 of the log, I saw that GitHub now issues subjects containing the immutable numeric IDs of the owner and repository. I added `github_owner_id` and `github_repo_id` variables, rebuilt the subject in exactly that shape, and the next run logged in. I consider this a better outcome than the original design, because a repository that is deleted and recreated with the same name receives a new ID, so it cannot inherit access to my registry.

4) Step 3: Creating Key Vault Without Putting Secrets in Terraform

The simplest way to put secrets in Key Vault with Terraform is the `azurerm_key_vault_secret` resource, and that is what my proposal draft assumed. I rejected it during implementation because Terraform records every attribute of every resource in its state file in plain text, including the secret value, so the vault would be secure while `terraform.tfstate` became a second, unprotected copy of every password. Instead, Terraform creates the vault empty and only defines who may use it:

```

resource "azurerm_key_vault" "kv" {
  ...
  rbac_authorization_enabled = false # pinned: newer API versions default to RBAC

  access_policy {
    # the operator running Terraform
    object_id      = data.azurerm_client_config.current.object_id
    secret_permissions = ["Get", "List", "Set", "Delete", "Purge", "Recover"]
  }
  access_policy {
    # the in-cluster workload identity
    object_id      = azurerm_user_assigned_identity.workload.principal_id
    secret_permissions = ["Get", "List"]
  }
}

```

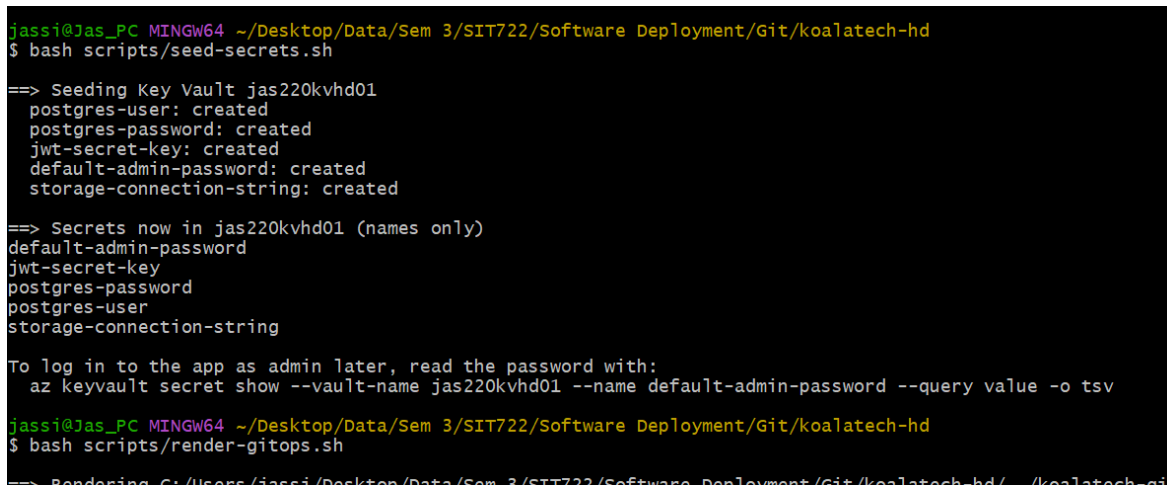
Excerpt from `terraform/key_vault.tf`.

The pods can only read. A compromised pod cannot change, delete or add a secret. I used access policies, the fallback I had approved in my proposal, because they are managed through the vault itself and needed no further role assignments on the CloudLabs subscription. I also had to pin `rbac_authorization_enabled = false` explicitly, because my first plan showed that value as known after apply, and newer Key Vault API versions create vaults in RBAC mode, which silently ignores access policies.

The values are then written by `scripts/seed-secrets.sh`. It generates each value with `openssl rand -hex`, sends it straight to Key Vault with the Azure CLI, and never prints it or writes it to a file. It is also idempotent, which matters more than it first appears, because PostgreSQL fixes its password the first time the data volume is initialised, so silently rotating that secret on a second run would lock every service out of its own database.

```
put_if_missing postgres-user      "koalatech"
put_if_missing postgres-password "${openssl rand -hex 24}"
put_if_missing jwt-secret-key     "${openssl rand -hex 32}"
put_if_missing default-admin-password "${openssl rand -hex 12}"
put_if_missing storage-connection-string "${az storage account show-connection-string ...}"
```

Excerpt from `scripts/seed-secrets.sh`. `put_if_missing` skips any secret that already exists.



```
jassi@Jas_PC MINGW64 ~/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech-hd
$ bash scripts/seed-secrets.sh

==> Seeding Key Vault jas220kvhd01
postgres-user: created
postgres-password: created
jwt-secret-key: created
default-admin-password: created
storage-connection-string: created

==> Secrets now in jas220kvhd01 (names only)
default-admin-password
jwt-secret-key
postgres-password
postgres-user
storage-connection-string

To log in to the app as admin later, read the password with:
az keyvault secret show --vault-name jas220kvhd01 --name default-admin-password --query value -o tsv

jassi@Jas_PC MINGW64 ~/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech-hd
$ bash scripts/render-gitops.sh

==> Rendering C:/Users/jassi/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech-hd/~/koalatech-hd
```

Figure 6: `seed-secrets.sh` creating the five secrets in `jas220kvhd01` and listing them by name only, followed by `render-gitops.sh` writing the Terraform outputs into the GitOps manifests.

Figure 6 shows the script creating all five secrets and then listing them by name only. Nothing in the output, in my shell history or in the repository contains a value. The next command, `render-gitops.sh`, copies the non secret Terraform outputs, the vault name, tenant ID and workload identity client ID, into the GitOps manifests, so the two repositories always agree with the infrastructure that was actually built.

5) Step 4: Delivering Secrets to the Pods as Files

With the identity and the vault in place, the cluster needs to know which secrets to fetch. That is the job of a `SecretProviderClass` in the GitOps repository:

```
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata: { name: koalatech-keyvault, namespace: production }
spec:
  provider: azure
  parameters:
    usePodIdentity: "false"
    clientId: "2acb228b-..." # workload identity, an identifier not a secret
    keyvaultName: "jas220kvhd01"
    objects: |
      array:
        - | { objectName: postgres-password, objectType: secret }
        ... # five objects in total
  # deliberately NO secretObjects block
```

Excerpt from `koalatech-gitops/manifests/02-secret-provider-class.yaml`, condensed.

When a pod that uses this class starts, the CSI driver takes the pod's projected ServiceAccount token, exchanges it with Entra ID for the workload identity's Azure token, reads the listed secrets from Key Vault, and writes each one

to an in memory volume at `/mnt/secrets-store`. The design decision I am most confident about is the missing `secretObjects` block. My original design included one, because most tutorials do, but it instructs the driver to copy every value into a Kubernetes Secret object, which would have recreated exactly the base64 secret I was trying to remove. Without it, the values exist only as files inside running pods.

The remaining problem was that the application reads its configuration from environment variables, and I did not want to change application code for an infrastructure feature. I solved it differently for the two kinds of container:

- **PostgreSQL** already supports this pattern. The official image reads `POSTGRES_USER_FILE` and `POSTGRES_PASSWORD_FILE` natively, so the database manifests simply point those variables at the mounted files.
- **The five FastAPI services** start through a short shell wrapper that reads each file into the environment of the `uvicorn` process only, then replaces itself with `uvicorn` using `exec`:

```
command: ["/bin/sh", "-c"]
args:
- |
  export POSTGRES_USER="$(cat /mnt/secrets-store/postgres-user)"
  export POSTGRES_PASSWORD="$(cat /mnt/secrets-store/postgres-password)"
  export JWT_SECRET_KEY="$(cat /mnt/secrets-store/jwt-secret-key)"
  export DEFAULT_ADMIN_PASSWORD="$(cat /mnt/secrets-store/default-admin-password)"
  exec uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Excerpt from `koalatech-gitops/manifests/20-user-service.yaml`. The other four services use the same wrapper.

Every Deployment also carries the label `azure.workload.identity/use: "true"` and runs as `serviceAccountName: koalatech-sa`, which is what makes the Workload Identity webhook inject the token. The ServiceAccount itself is annotated with the workload identity client ID, so the token is exchanged for the right identity. The evidence that this works end to end is in Section 9, Figures 16 to 18, where the running pod holds the real password and the application logs in with it.

6) Steps 5 and 6: The GitOps Repository and Argo CD

The second repository, `koalatech-gitops`, is the single source of truth for production. It contains the namespace, the ServiceAccount, the SecretProviderClass, five databases, five services and the frontend as plain manifests, plus a `kustomization.yaml` whose `images` block holds the tag of every image. CI later changes only that `images` block. The repository also holds one Argo CD Application, which is the only manifest I ever apply by hand:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: koalatech-production, namespace: argocd }
spec:
  source:
    repoURL: https://github.com/JasSin155/koalatech-gitops.git
    targetRevision: main
    path: manifests
  destination: { server: https://kubernetes.default.svc, namespace: production }
  syncPolicy:
    automated:
      prune: true      # a manifest deleted from Git is deleted from the cluster
      selfHeal: true   # a manual kubectl change is reverted to match Git
    retry: { limit: 5, backoff: { duration: 10s, factor: 2, maxDuration: 3m } }
```

Excerpt from `koalatech-gitops/argocd/koalatech-production.yaml`.

`prune` and `selfHeal` are what turn Argo CD from a deployment tool into an enforcement tool. Without them, Argo CD would deploy Git changes but would leave manual changes and deleted resources in place, which is the same weakness as the push pipeline. The `retry` policy covers the first sync, when the databases are still starting and the services briefly fail their readiness probes.

scripts/bootstrap-argocd.sh installs Argo CD and registers the Application. I pinned Argo CD to v3.5.3 rather than following the stable branch, so that the steps in this report and the video still produce the same result later. I also had to install it with `kubectl apply --server-side`, because the Argo CD custom resource definitions are larger than the annotation that client side apply writes, and the ordinary apply fails on them. After this script, no command in either repository applies anything to production again.

```
jassi@Jas_PC MINGW64 ~/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech
$ kubectl get pods -n production
NAME                                READY   STATUS    RESTARTS   AGE
course-db-76669c4b8c-cfvpv          1/1     Running   0           2m38s
course-service-6d9d78d5b4-kjf8s     1/1     Running   0           2m38s
enrollment-db-78784b8886-j5k5m      1/1     Running   0           2m39s
enrollment-service-5d569bd879-26927 1/1     Running   0           2m38s
frontend-74679457d-g7s5m            1/1     Running   0           2m38s
frontend-74679457d-17srn            1/1     Running   0           2m38s
lecturer-db-55bf6db7fd-w7mws        1/1     Running   0           2m39s
lecturer-service-58fcbd85bd-nvg57   1/1     Running   0           2m39s
student-db-6849b6947f-flq95         1/1     Running   0           2m38s
student-service-68488fb59b-r19tc     1/1     Running   0           2m39s
user-db-ff7c4cc9c-9xhk6             1/1     Running   0           2m38s
user-service-769677ccbc-trjgz       1/1     Running   0           2m39s

jassi@Jas_PC MINGW64 ~/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech
$ kubectl get application -n argocd koalatech-production
NAME                                SYNC STATUS  HEALTH STATUS
koalatech-production              Synced      Healthy

jassi@Jas_PC MINGW64 ~/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech
$
```

Figure 7: All twelve production pods Running with no restarts, and the koalatech-production Application reporting Synced and Healthy.

Figure 7 shows the first sync. Every pod is Running with zero restarts, which is meaningful here because a database that could not read its password file, or a service that could not reach its database, would restart repeatedly. The Application status confirms that what is running matches Git exactly.

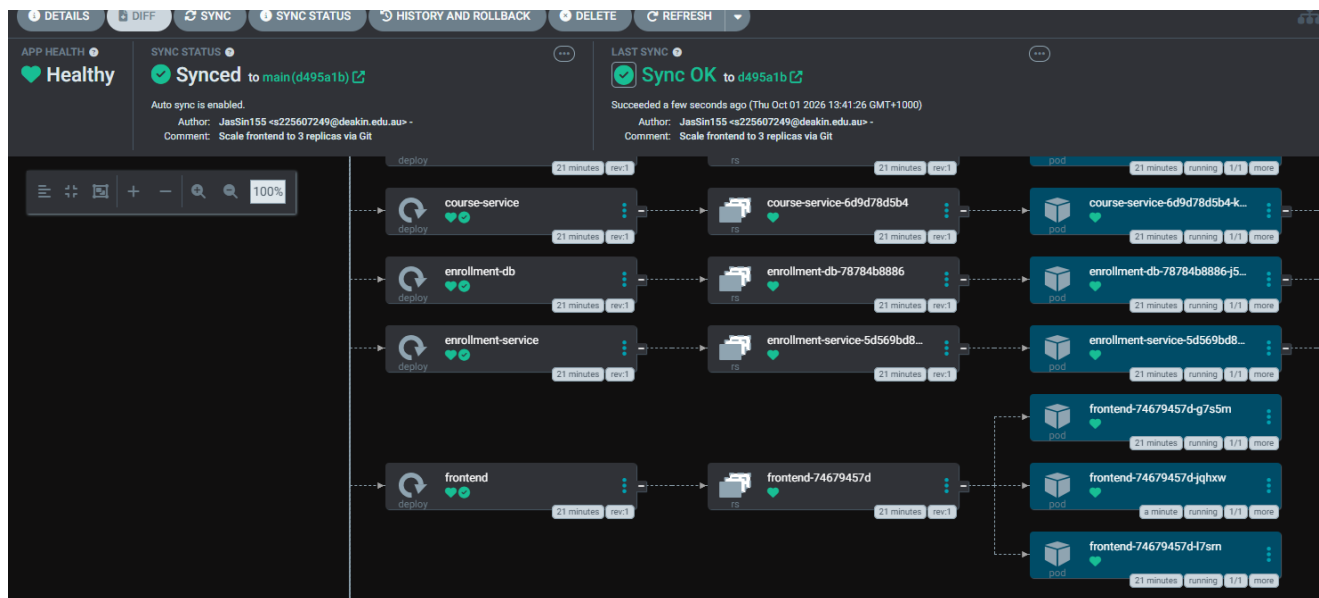


Figure 8: The Argo CD resource tree for koalatech-production: Healthy, Synced to main (d495a1b), with auto sync enabled and the commit author and message shown.

Figure 8 is the Argo CD view of the same application. The header shows that the cluster is synced to a specific Git commit, with its author and message, which is the audit trail that `kubectl apply` never gave me. Each Deployment fans out to its ReplicaSet and pods, and each pod carries a green health marker. The commit shown, d495a1b, is the replica change from Section 9, which is why the frontend has three pods.

7) Step 7: Connecting CI to GitOps Without Giving It Cluster Access

The workflow in `.github/workflows/ci.yml` has four jobs. checkov and the unit tests run first, then six parallel build and scan jobs, then a single hand off job. On a pull request the build jobs stop after scanning. On a push to main they log in to Azure with the federated identity from Section 3, exchange the Azure token for a registry token through the ACR `/oauth2/exchange` endpoint, and push the image tagged with the commit SHA. I used the token exchange rather than `az acr login` because it needs nothing beyond the `AcrPush` role, and the token is masked in the log.

The last job is where the push model ends. It checks out the GitOps repository, updates the image tags with kustomize, and commits:

```
- name: Set new image tags
  working-directory: gitops/manifests
  run: |
    for img in frontend user-service student-service \
      lecturer-service course-service enrollment-service; do
      kustomize edit set image "koalatech-${img}=${ACR}/koalatech-${img}:${TAG}"
    done

- name: Commit and push
  run: |
    git config user.name "koalatech-ci[bot]"
    git add manifests/kustomization.yaml
    git commit -m "deploy: koalatech images ${GITHUB_SHA::7} \
      (from ${GITHUB_REPOSITORY}@${GITHUB_SHA::7})"
    git push
```

Excerpt from the update-gitops job in `.github/workflows/ci.yml`.

The commit message links each deployment back to the exact application commit that produced it, so either repository's history can be traced to the other. The job runs in a `gitops-write` concurrency group, so two quick pushes cannot race each other to the GitOps repository.

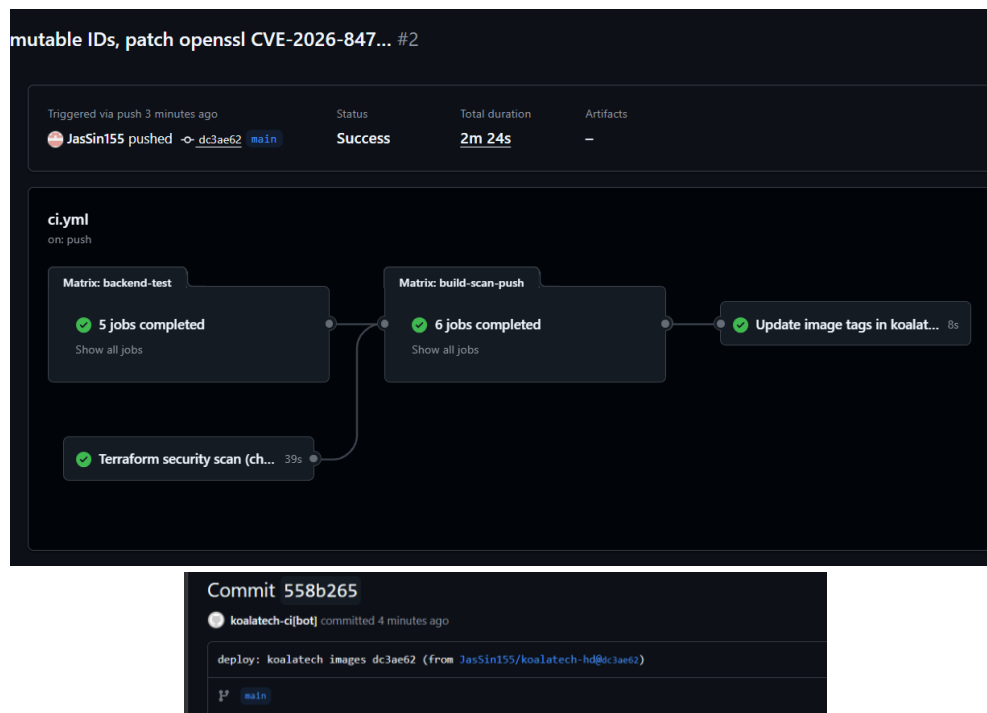


Figure 9: Top: pipeline run #2 succeeding on commit `dc3ae62`, with checkov, five test jobs, six build and scan jobs and the GitOps hand off. Bottom: the resulting commit `558b265` by `koalatech-ci[bot]` in `koalatech-gitops`.

The top of Figure 9 shows the full pipeline in 2 minutes 24 seconds, with the job graph making the order of the gates visible: no image is built until checkov and the tests have passed. The bottom shows the only effect CI has on production, a single commit by the bot account, whose message references dc3ae62, the application commit that triggered the run.

```

33 - newname: jas228acrhd.azurecr.io/koalatech-student-service
34 - newtag: bootstrap
35 - name: koalatech-lecturer-service
36 - newname: jas228acrhd.azurecr.io/koalatech-lecturer-service
37 - newtag: bootstrap
38 - name: koalatech-course-service
39 - newname: jas228acrhd.azurecr.io/koalatech-course-service
40 - newtag: bootstrap
41 - name: koalatech-enrollment-service
42 - newname: jas228acrhd.azurecr.io/koalatech-enrollment-service
43 - newtag: bootstrap

26 + name: koalatech-course-service
27 + newname: jas228acrhd.azurecr.io/koalatech-course-service
28 + newtag: dc3ae62aa906420bbb162f22bd02634f8a408862
29 + name: koalatech-enrollment-service
30 + newname: jas228acrhd.azurecr.io/koalatech-enrollment-service
31 + newtag: dc3ae62aa906420bbb162f22bd02634f8a408862
32 + name: koalatech-frontend
33 + newname: jas228acrhd.azurecr.io/koalatech-frontend
34 + newtag: dc3ae62aa906420bbb162f22bd02634f8a408862
35 + name: koalatech-lecturer-service
36 + newname: jas228acrhd.azurecr.io/koalatech-lecturer-service
37 + newtag: dc3ae62aa906420bbb162f22bd02634f8a408862
38 + name: koalatech-student-service
39 + newname: jas228acrhd.azurecr.io/koalatech-student-service
40 + newtag: dc3ae62aa906420bbb162f22bd02634f8a408862

```

Figure 10: The diff of that bot commit: every image moves from the bootstrap tag to dc3ae62aa906420bbb162f22bd02634f8a408862.

Figure 10 shows the diff itself. The only lines that change are the image tags, from bootstrap to the full commit SHA, which is why a reviewer can read any deployment in a few seconds. Argo CD detected this commit and rolled out the new images, which is the state shown in Figures 7 and 8.

8) Step 8: Making the Security Gates Blocking

In Task 10.1P I ran Docker Scout by hand, and in Task 10.2D it ran in the pipeline, but nothing prevented a failing change from being merged. For this task I built two gates and then made them compulsory.

Gate 1: checkov on the Terraform module

checkov 3.3.20 scans every Terraform file on each push and pull request. Its first run against my module reported 35 findings. Rather than suppress them in bulk, I reviewed each one. I fixed two in code, blob soft delete (CKV2_AZURE_38) and a SAS expiry policy (CKV2_AZURE_41), and accepted the remaining 33 in .checkov.yaml, each with a written reason, for example:

```

skip-check:
- CKV_AZURE_166 # quarantine: Premium only, Docker Scout scans images in CI instead
- CKV_AZURE_110 # purge protection: off so teardown can purge the vault the same day
- CKV_AZURE_141 # local admin: needs Entra ID admin groups, not available on CloudLabs

```

Three of the 33 documented skips in .checkov.yaml.

This makes the configuration itself the record of accepted risk, in the same way a risk register records why a control is not applied. Any finding not on the list fails the job, and because every build job declares needs: [terraform-scan, backend-test], nothing is built or pushed after a checkov failure.

Gate 2: Docker Scout on every image

```
- name: Docker Scout gate
  uses: docker/scout-action@v1
  with:
    command: cves
    image: ${ env.IMAGE_REF }
    only-severities: critical,high
    only-fixed: "true"
    exit-code: "1"
```

The Docker Scout step in the build-scan-push job, which runs before any login or push.

I kept the settings I had proven in Task 10.2D: fail on critical or high findings that have a fix available. only-fixed keeps the gate actionable, because without it the build would block indefinitely on operating system vulnerabilities that Debian has not yet patched.

```
27
28 Target      | ***acrhd.azurecr.io/koalatech-user-service:a97a56e9fb2a2130423de3accaf5901fdeb9de6d
29 digest      | 8b211812a662
30 platform    | linux/amd64
31 vulnerabilities | 0C 1H 0M 0L
32 size        | 86 MB
33 packages    | 163
34
35
36 ## Packages and Vulnerabilities
37
38 0C 1H 0M 0L openssl 3.5.7-1~deb13u2
39 pkg:deb/debian/openssl@3.5.7-1~deb13u2?os_distro=trixie&os_name=debian&os_version=13
40
41 X HIGH CVE-2026-84782
42 https://scout.docker.com/v/CVE-2026-84782?s=debian&n=openssl&n=debian&t=deb&osn=debian&osv=13&vr=%!C(string= )3.5.7-1%!E(MISSING)deb13u3
43 Affected range : <3.5.7-1~deb13u3
44 Fixed version  : 3.5.7-1~deb13u3
```

Figure 11: Pipeline run #1: Docker Scout failing koalatech-user-service on CVE-2026-84782 (high) in openssl 3.5.7-1~deb13u2, with a fixed version, 3.5.7-1~deb13u3, available.

The gate proved itself on its very first run. Figure 11 shows Docker Scout blocking the user service image on CVE-2026-84782, a high severity openssl vulnerability in the Debian 13 base layer, and the same finding blocked all five backend images. The fix was available upstream but not yet in the python:3.12-slim image I build from. I fixed the image rather than weakening the gate, by applying Debian security updates in every backend Dockerfile:

```
FROM python:3.12-slim
RUN apt-get update \
    && apt-get upgrade -y --no-install-recommends \
    && rm -rf /var/lib/apt/lists/*
```

Added to the five backend Dockerfiles in commit dc3ae62. Run #2 passed the gate.

Making the gates compulsory

A failing check on its own is only advice, because anyone with write access can still press merge. I created a branch ruleset, protect-main, that requires a pull request and requires these checks to pass before merging. To test it, I opened pull request #1, which changed the storage account's minimum TLS version to TLS 1.0, a realistic misconfiguration that would weaken encryption in transit.

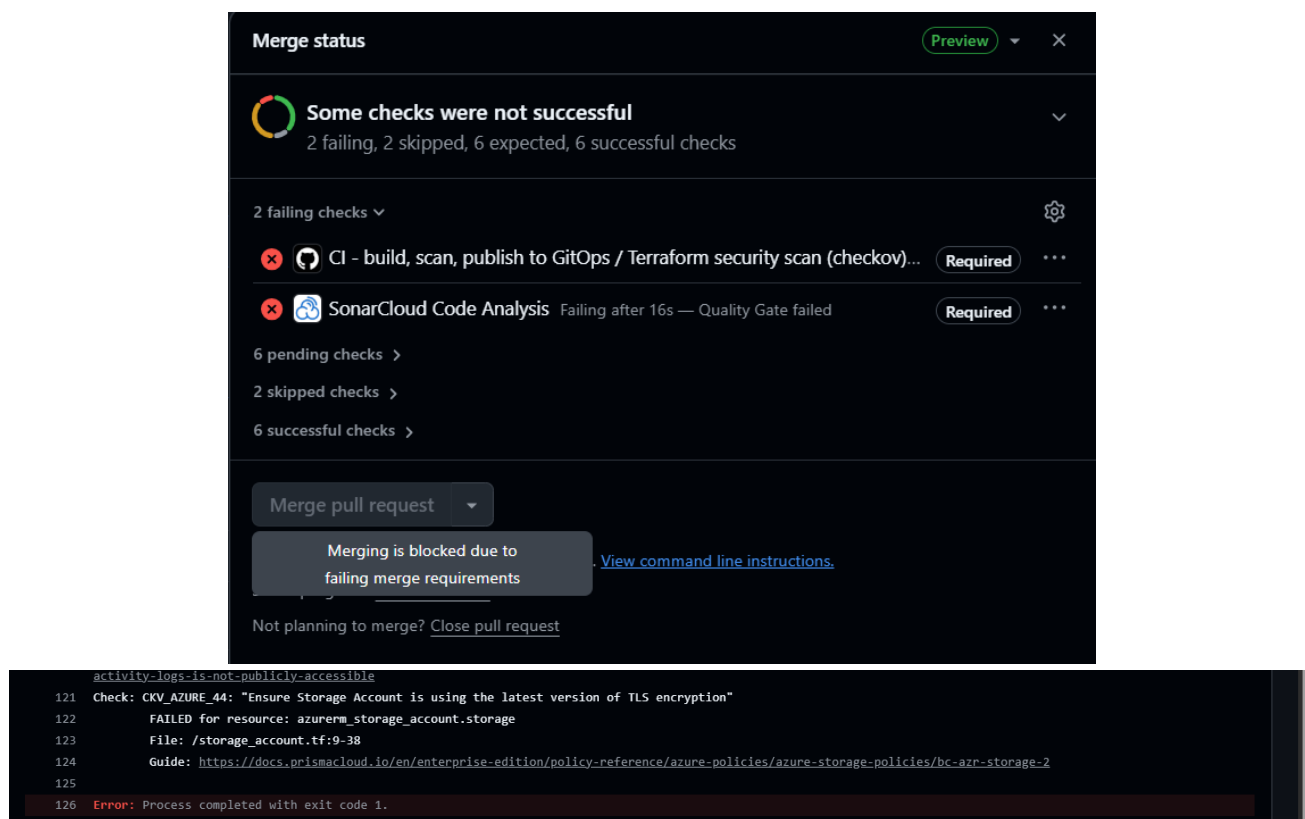


Figure 12: Pull request #1: checkov and SonarQube Cloud both fail, the six build and scan checks never start, and merging is blocked (top). The checkov log identifies CKV_AZURE_44 at storage_account.tf lines 9 to 38 (bottom).

Figure 12 shows the result. checkov failed the pull request on CKV_AZURE_44, Ensure Storage Account is using the latest version of TLS encryption, and named the exact file and lines. Because the build jobs depend on checkov, the six image checks never ran, and the merge button is disabled with "Merging is blocked due to failing merge requirements". SonarQube Cloud, which I had connected to my GitHub account earlier in the unit, independently failed its quality gate on the same change. Two separate tools reaching the same verdict gave me confidence that the gate was detecting a real problem rather than a configuration quirk.

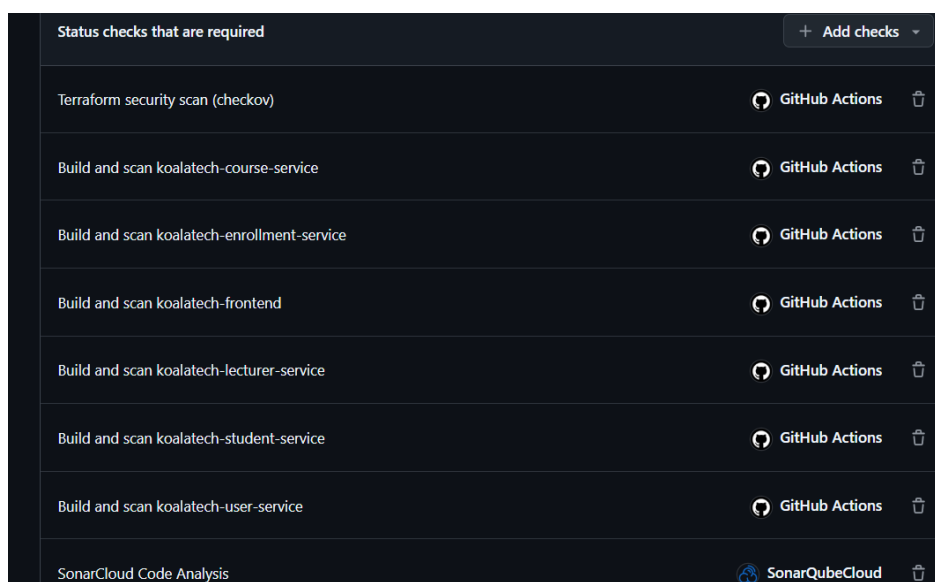


Figure 13: The protect-main ruleset's required status checks: checkov, the six build and scan jobs, and SonarCloud Code Analysis.

Figure 13 shows the eight required checks. I required each build and scan job individually, rather than the workflow as a whole, so that a vulnerability in any single image is enough to block a merge.

9) Verification Against My Success Criteria

My Task 7.3HD proposal committed to three measurable outcomes. I tested each one on the live system rather than inferring it from configuration.

Criterion 1: a Git commit reaches the cluster with no pipeline and no kubectl apply

```
jassi@Jas_PC MINGW64 ~/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech-gitops
$ git commit -am "Scale frontend to 3 replicas via Git"
git push
[main d495a1b] Scale frontend to 3 replicas via Git
1 file changed, 1 insertion(+), 1 deletion(-)
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 22 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 371 bytes | 371.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/JasSin155/koalatech-gitops.git
558b265..d495a1b main -> main

jassi@Jas_PC MINGW64 ~/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech-gitops
$ kubectl get pods -n production -l app=frontend
NAME                                READY   STATUS    RESTARTS   AGE
frontend-74679457d-g7s5m            1/1     Running   0           19m
frontend-74679457d-l7srn            1/1     Running   0           19m

jassi@Jas_PC MINGW64 ~/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech-gitops
$ kubectl get pods -n production -l app=frontend
NAME                                READY   STATUS    RESTARTS   AGE
frontend-74679457d-g7s5m            1/1     Running   0           20m
frontend-74679457d-jqhxw            1/1     Running   0           30s
frontend-74679457d-l7srn            1/1     Running   0           20m
```

Figure 14: Changing the frontend replica count in koalatech-gitops and pushing commit d495a1b. Thirty seconds later a third frontend pod is Running, with no workflow run and no kubectl apply.

In Figure 14 I edited one line in manifests/30-frontend.yaml and pushed it straight to the GitOps repository. The first `kubectl get pods` shows the original two frontend pods, and the second, taken shortly afterwards, shows a third pod aged 30 seconds. No GitHub Actions workflow ran, and I did not apply anything to the cluster, so Argo CD's reconciliation is the only explanation for the new pod. In the video I demonstrated the same mechanism in the other direction, scaling from three to two.

```
jassi@Jas_PC MINGW64 ~/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech-gitops
$ kubectl scale deployment frontend -n production --replicas=1
kubectl get deployment frontend -n production -w
deployment.apps/frontend scaled
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
frontend  1/1     1            1           22m
frontend  1/1     1            1           22m
frontend  1/3     1            1           22m
frontend  1/3     1            1           22m
frontend  1/3     1            1           22m
frontend  1/3     3            1           22m
frontend  1/3     3            1           22m
frontend  1/3     3            1           22m
frontend  2/3     3            2           22m
frontend  3/3     3            3           22m
```

Figure 15: A manual kubectl scale to one replica being reverted by selfHeal: READY moves from 1/1 back to 3/3 within seconds.

Figure 15 tests the reverse case, which a push pipeline cannot handle. I scaled the frontend to one replica with kubectl, the kind of manual fix someone might make during an incident. The watch output shows the Deployment at 1/1, then Argo CD restoring the desired count from Git, and the replicas returning to 3/3. The cluster cannot drift away from Git for longer than one reconciliation.

Criterion 2: a pod receives a Key Vault secret that exists nowhere in Git or Terraform state

I wrote `scripts/verify-no-secrets.sh` so that this claim is tested rather than asserted. It reads the real PostgreSQL password from Key Vault into a shell variable without printing it, then searches for that exact value in every commit of both repositories, in `terraform.tfstate`, and among Kubernetes Secrets, and finally compares it with the file inside a running pod.

```
jassi@Jas_PC MINGW64 ~/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech-hd (main)
$ bash scripts/verify-no-secrets.sh
Fetched postgres-password from jas220kvhd01 (48 characters, value not shown)

==> 1. App repo, every commit on every branch
    matches: 0

==> 2. GitOps repo, every commit on every branch
    matches: 0

==> 3. terraform.tfstate
    matches: 0

==> 4. Kubernetes Secret objects in the production namespace
    Secret objects: 0

==> 5. But the running pod DOES have it, mounted from Key Vault
total 0
lrwxrwxrwx 1 root root 29 Oct 1 03:20 default-admin-password -> ..data/default-admin-password
lrwxrwxrwx 1 root root 21 Oct 1 03:20 jwt-secret-key -> ..data/jwt-secret-key
lrwxrwxrwx 1 root root 24 Oct 1 03:20 postgres-password -> ..data/postgres-password
lrwxrwxrwx 1 root root 20 Oct 1 03:20 postgres-user -> ..data/postgres-user
lrwxrwxrwx 1 root root 32 Oct 1 03:20 storage-connection-string -> ..data/storage-connection-string
pod file matches Key Vault value: yes

jassi@Jas_PC MINGW64 ~/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech-hd (main)
$ |
```

Figure 16: `verify-no-secrets.sh`: zero matches in every commit of both repositories and in `terraform.tfstate`, zero Secret objects, five secret files mounted in the pod, and the pod's value matching Key Vault.

Figure 16 shows each check. The password is 48 characters, so an accidental match is not plausible, and searching `git log --all -p` covers every version of every file on every branch, not just the current one. Step 5 then shows the other half of the claim: the pod does hold the value. The five files in `/mnt/secrets-store` are symbolic links into the CSI driver's `..data` directory, which is the in memory volume the driver manages, and the pod's copy of the password matches Key Vault.

```
jassi@Jas_PC MINGW64 ~/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech-hd (main)
$ kubectl get secrets -n production
No resources found in production namespace.

jassi@Jas_PC MINGW64 ~/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech-hd (main)
$ |
```

Figure 17: `kubectl get secrets -n production` returning No resources found.

Figure 17 confirms the same point from Kubernetes itself. There is not a single Secret object in the production namespace, which is only possible because I removed the `secretObjects` block in Section 5.

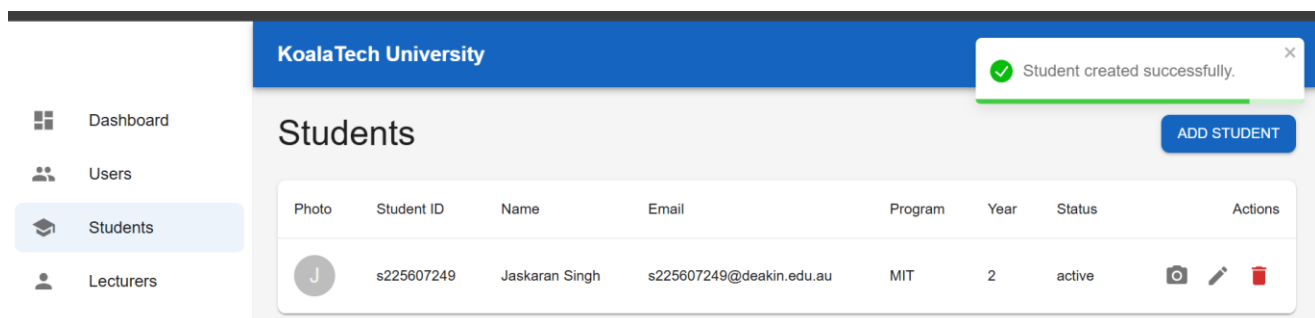


Figure 18: The KoalaTech application in production after logging in as admin with the Key Vault generated password, creating a student record successfully.

Figure 18 closes the loop from the user's side. I logged in as admin with the password generated by the seed script, which I read from Key Vault and never stored anywhere, and created a student record. The login proves the user service read its admin password and JWT key from the mount, and the saved record proves the student service and its database agree on credentials that exist only in Key Vault.

Criterion 3: CI blocks a build on a misconfiguration or a vulnerability

Both cases occurred, as described in Section 8. Docker Scout blocked five images on a real openssl vulnerability on the first run (Figure 11), and checkov blocked a pull request that set TLS 1.0, with the ruleset disabling the merge (Figure 12). The first of these was not staged, which I consider stronger evidence than the planned test, because it shows the gate catching a vulnerability I did not know existed.

10) Problems I Encountered and How I Resolved Them

Most of what I learnt in this task came from things that did not work the first time. Table 2 lists each problem, the cause I found, and the fix that is now in the repositories, with a comment beside the fix in the code.

Problem	Cause	Fix
Azure login failed with AADSTS700213	GitHub now issues OIDC subjects with numeric owner and repository IDs	Pinned the exact subject from the error, which is also stricter than a name based subject
Docker Scout blocked all five backend images	CVE-2026-84782 in the Debian openssl package of the base image	apt-get upgrade in each Dockerfile, fixing the image rather than loosening the gate
seed-secrets.sh could not find terraform.tfstate	MSYS_NO_PATHCONV=1 in Git Bash stopped /c/... paths being translated for the Windows terraform.exe	Resolved the repository root with pwd -W, which returns a Windows path
Plan showed the vault's authorisation mode as known after apply	Newer Key Vault API versions create vaults in RBAC mode, which ignores access policies	Pinned rbac_authorization_enabled = false
A second plan wanted to update the AKS node pool	Azure applies upgrade_settings defaults that the module did not declare	Declared the defaults explicitly, so plans are clean and no node update is triggered
Argo CD install failed with client side apply	Its CRDs exceed the size limit of the last-applied annotation	kubectl apply --server-side --force-conflicts
SARIF upload found no file, and Node 20 deprecation warnings	Wrong output file name, and older action versions	Wrote results.sarif explicitly and moved every action to its Node 24 release
terraform.tfvars was ignored by Git	The week08 .gitignore excluded every *.tfvars file	Added an exception, which is safe because the file now holds only resource names

Table 2. Problems encountered during implementation.

11) Limitations and Further Work

I want to be precise about what this design does not achieve, because an inflated claim would undermine the evidence above.

- **The storage account key is in Terraform state.** Azure generates it and the azurerm provider records it as an attribute of the storage account. My verification searched for the PostgreSQL password, so the zero secrets result applies to every secret I create. I kept the state file local and gitignored, and teardown deletes it. A production version would use a remote backend with encryption and restricted access, or disable shared key access and use identity based storage access.

- **Access policies rather than Azure RBAC on the vault.** This was the approved fallback and gives the same read only result for the pods, but RBAC would allow finer scoping, for example per secret, and is Microsoft's recommended model for new vaults.
- **One environment.** Argo CD manages production only. The natural next step is a staging Application on its own path or branch, with promotion to production by pull request, so that the same ruleset protects promotion.
- **Public endpoints.** Several of the accepted checkov findings exist because the sandbox has no private networking. Private endpoints for Key Vault and ACR would remove them.

12) Teardown

Once the video and this evidence were complete, I removed every resource the task created with scripts/teardown.sh. The script asks me to type DELETE, deletes the resource group, which also removes the AKS node resource group, then purges the soft deleted Key Vault so that its name and secrets do not linger for the seven day retention period, and finally deletes the local Terraform state and the kubectl context. I also closed pull request #1 without merging, deleted its branch, and revoked the GITOPS_TOKEN and Docker Hub tokens, since a token for a repository that no longer deploys anything should not stay valid.

```
jassi@Jas_PC MINGW64 ~/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech-hd (main)
$ cd "/c/Users/jassi/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech-hd"
bash scripts/teardown.sh
Type DELETE to remove resource group koalatech-hd-rg and everything in it: DELETE

==> Deleting koalatech-hd-rg (takes 5 to 10 minutes)

==> Purging soft-deleted Key Vault jas220kvhd01

==> Removing local Terraform state and kube context
deleted context jas220akshd from C:\Users\jassi\.kube\config

==> Checks (both should be empty / false)
false

jassi@Jas_PC MINGW64 ~/Desktop/Data/Sem 3/SIT722/Software Deployment/Git/koalatech-hd (main)
$ az group list --query "[?contains(name,'koalatech-hd')].name" -o tsv
$ az keyvault list-deleted --query "[].name" -o tsv
$ az role assignment list --assignee a3f6fa69-4ccf-4f71-946b-8cd18acd2c2ba --all --query "[].roleDefinitionName" -o tsv
ls terraform/*.tfstate* 2>/dev/null || echo "no local state left"
Reader
Contributor - Custom role-v2 I68641 S1 UcJLF
no local state left
```

Figure 19: teardown.sh deleting koalatech-hd-rg, purging the soft deleted vault and removing the local state and kube context, followed by checks: no koalatech-hd resource group, no soft deleted vault, only my baseline Reader and Contributor roles, and no local Terraform state.

Figure 19 shows the script and four independent checks. The resource group query and the deleted vault query both return nothing, so neither the resources nor a recoverable copy of the vault remain. My role list is back to the two CloudLabs baseline roles, because the User Access Administrator grant from Section 2 was scoped to the resource group and disappeared with it, which meant I had no standing access left to clean up by hand, unlike in Task 10.2D. Finally, no Terraform state file remains on my machine, which also removes the storage account key discussed in Section 11.

13) Generative AI Use

I used generative AI (Claude) as an assistant while designing and drafting the Terraform, manifests, scripts and workflow for this task, and to help structure this report. I ran every command myself and checked each suggestion against what I could actually observe, which is how I confirmed that my first design's azurerm_key_vault_secret resources would have stored every value in terraform.tfstate, and why I replaced them with an empty vault and a seed script. For each failure in Table 2, I confirmed the fix against the actual error output before applying it, for example by copying the exact OIDC subject from the AADSTS700213 message rather than trusting a suggested format.

References

Argo CD documentation, Automated sync policy. https://argo-cd.readthedocs.io/en/stable/user-guide/auto_sync/

Microsoft Learn, Use Microsoft Entra Workload ID with AKS. <https://learn.microsoft.com/en-us/azure/aks/workload-identity-overview>

Microsoft Learn, Use the Azure Key Vault provider for Secrets Store CSI Driver in AKS. <https://learn.microsoft.com/en-us/azure/aks/csi-secrets-store-driver>

GitHub Docs, Configuring OpenID Connect in Azure. <https://docs.github.com/en/actions/security-for-github-actions/security-hardening-your-deployments/configuring-openid-connect-in-azure>

HashiCorp, Sensitive data in state. <https://developer.hashicorp.com/terraform/language/state/sensitive-data>

Bridgecrew, checkov documentation. <https://www.checkov.io/>

Docker Docs, Docker Scout. <https://docs.docker.com/scout/>